

## **ACA – FOODEXPRESS**

### **Sistema web para gestión de pedidos de restaurante**

Andres Felipe Arevalo Martin

Ingeniería Web - 54488

Escuela de Ingeniería, Corporación Universitaria Nacional

Ing. Andres Camilo Ortiz Cogollo

13 de septiembre de 2026

## Contenido

|                    |   |
|--------------------|---|
| Introducción ..... | 3 |
|--------------------|---|

## **Introducción**

FOODEXPRESS es una aplicación web orientada a la gestión de pedidos de un restaurante. El proyecto surge de la necesidad de modernizar procesos que, cuando se realizan manualmente o mediante herramientas poco integradas, pueden generar demoras en la atención, errores en el registro de pedidos y dificultades para conocer el estado de las órdenes. La solución integra un frontend desarrollado con React, una API REST construida con Node.js y Express, y persistencia de datos en MongoDB mediante Mongoose. El desarrollo aplica conceptos de Ingeniería Web relacionados con arquitectura por componentes, servicios REST, persistencia NoSQL, control de versiones y contenerización.

## **Objetivos**

### **Objetivo General**

Diseñar y desarrollar una aplicación web denominada FOODEXPRESS, aplicando principios de Ingeniería Web para construir una solución orientada a servicios que permita administrar los principales procesos de un restaurante.

### **Objetivos Específicos**

- Diseñar una arquitectura web basada en componentes.
- Implementar una API REST para la comunicación entre frontend y backend.
- Implementar persistencia de datos NoSQL mediante MongoDB.
- Desarrollar un frontend que permita consultar productos y gestionar un carrito.
- Registrar y consultar pedidos.
- Permitir la actualización del estado de los pedidos.
- Gestionar el código fuente mediante Git y GitHub.
- Preparar la infraestructura de contenerización mediante Dockerfile y Docker Compose.

### **Planteamiento del problema**

El restaurante requiere modernizar su proceso de gestión de pedidos. La operación manual o mediante herramientas poco integradas puede ocasionar demoras, errores en el registro de las órdenes y dificultades para controlar su estado. FOODEXPRESS busca centralizar la información relacionada con productos, categorías, clientes, usuarios y pedidos mediante una aplicación web que permita consultar el menú, registrar información y gestionar el ciclo básico de un pedido.

### **Alcance**

El sistema contempla cinco entidades principales: productos, categorías, clientes, pedidos y usuarios. El MVP implementado prioriza el flujo extremo a extremo de consulta de productos, búsqueda, carrito, creación de pedidos, persistencia y actualización de estados.

Las operaciones administrativas complementarias fueron implementadas principalmente mediante la API REST y validadas con Postman.

## **Actores**

### **Cliente:**

Consulta el menú, busca productos, agrega productos al carrito, modifica cantidades y realiza pedidos.

### **Administrador:**

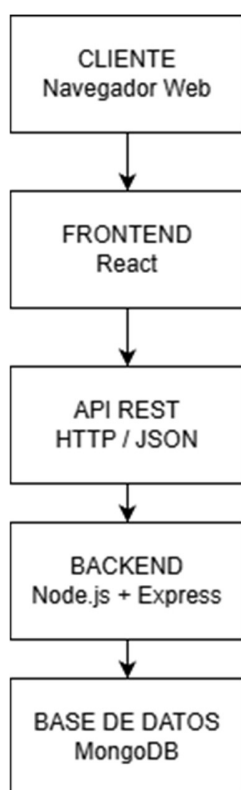
Gestiona información del sistema, consulta pedidos y actualiza su estado.

## **Estado de implementación**

La entrega presenta un MVP de FOODEXPRESS centrado en el flujo principal de productos y pedidos. Se implementó una API REST en Node.js y Express con persistencia MongoDB mediante Mongoose, un frontend React que consulta productos, administra un carrito y registra pedidos, y una vista administrativa mínima para consultar y actualizar estados. Las operaciones complementarias fueron verificadas mediante Postman. Debido al tiempo de desarrollo disponible se priorizó funcionalidad demostrable sobre acabado visual y cobertura exhaustiva de todas las interfaces administrativas.

## **Arquitectura**

Los productos pertenecen a categorías; los pedidos se relacionan con un cliente y contienen uno o varios productos con cantidad y precio unitario.



### Api Rest

| Método        | Ruta                      | Función             |
|---------------|---------------------------|---------------------|
| <b>GET</b>    | /api/productos            | Consultar y buscar  |
| <b>POST</b>   | /api/productos            | Crear producto      |
| <b>PUT</b>    | /api/productos/:id        | Actualizar producto |
| <b>DELETE</b> | /api/productos/:id        | Eliminar producto   |
| <b>GET</b>    | /api/categorias           | Consultar categoría |
| <b>POST</b>   | /api/clientes             | Registrar cliente   |
| <b>POST</b>   | /api/auth/admin/login     | Login Admin         |
| <b>POST</b>   | /api/pedidos              | Registrar pedido    |
| <b>GET</b>    | /api/pedidos              | Consultar pedidos   |
| <b>GET</b>    | /api/pedidos/:id          | Consultar detalles  |
| <b>PUT</b>    | /api/pedidos/:id/estado   | Cambiar estado      |
| <b>PUT</b>    | /api/pedidos/:id/cancelar | Cancelar pedido     |

### Requerimientos implementados

Entre los requerimientos funcionales demostrados se encuentran la creación y consulta de productos y categorías, registro de clientes, autenticación del administrador, consulta del menú, búsqueda de productos, gestión del carrito, cálculo del total, registro de pedidos, consulta de pedidos y actualización de su estado.

### Pruebas

| Código | Prueba                      | Resultado |
|--------|-----------------------------|-----------|
| P01    | Crear categoría             | OK        |
| P02    | Crear producto              | OK        |
| P03    | Consultar productos         | OK        |
| P04    | Registrar cliente           | OK        |
| P05    | Login administrador         | OK        |
| P06    | Agregar producto al carrito | OK        |
| P07    | Modificar cantidad y total  | OK        |
| P08    | Crear pedido                | OK        |
| P09    | Consultar pedidos           | OK        |
| P10    | Cambiar estado del pedido   | OK        |

### DOCKER

Se prepararon los archivos backend/Dockerfile, frontend/Dockerfile y docker-compose.yml para la contenerización de MongoDB, backend y frontend.

Durante la prueba final, el comando docker no fue reconocido por el entorno de desarrollo utilizado. Por esta razón no se afirma que la ejecución mediante contenedores haya sido validada. La demostración funcional se realizó mediante ejecución local del backend y frontend y conexión con MongoDB Atlas.

## Evidencias

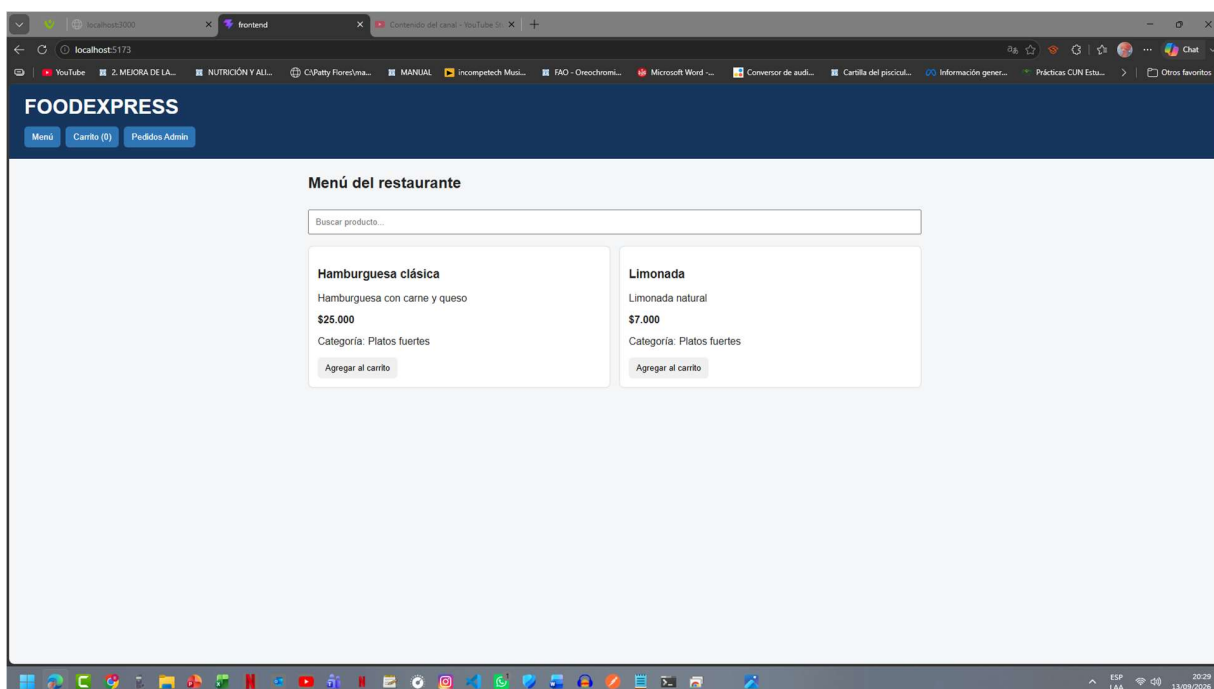


Figura 1. Productos recuperados desde la API REST y mostrados en React.

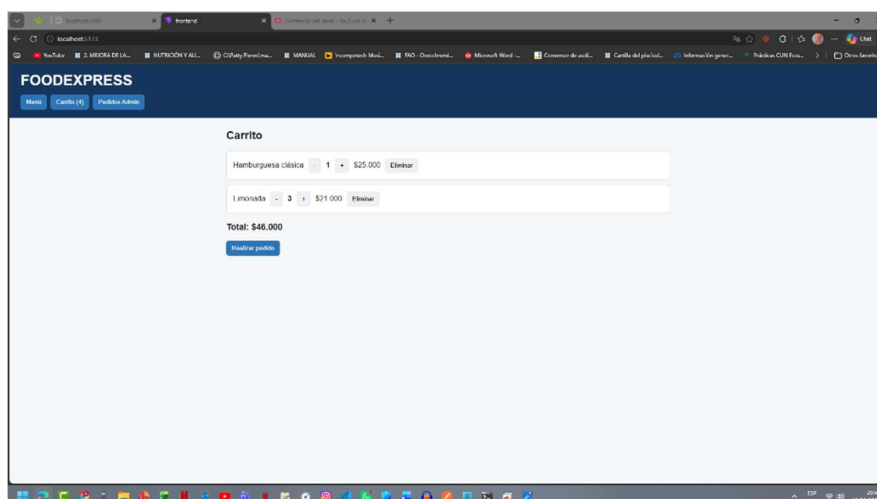


Figura 2. Carrito de compras con productos, cantidades y cálculo automático del total.

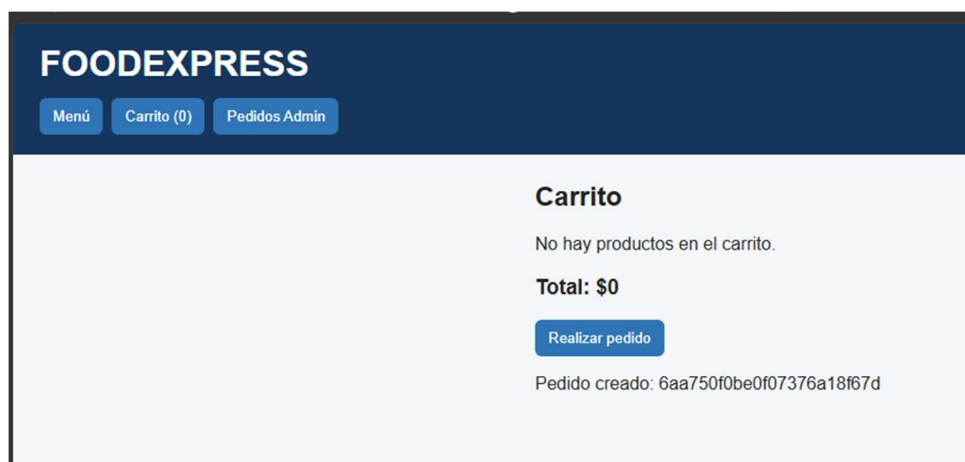


Figura 3. Confirmación de creación de un pedido mediante la aplicación web.

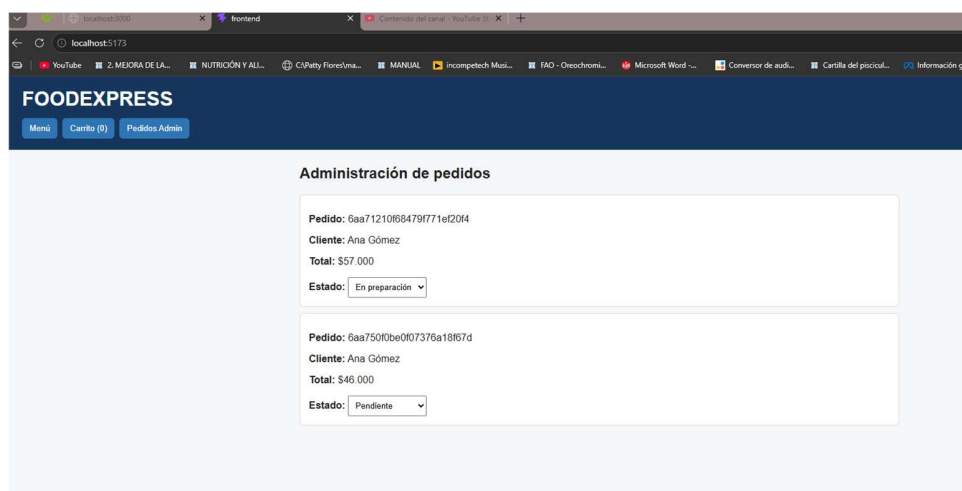


Figura 4. Vista administrativa de pedidos con actualización del estado del pedido

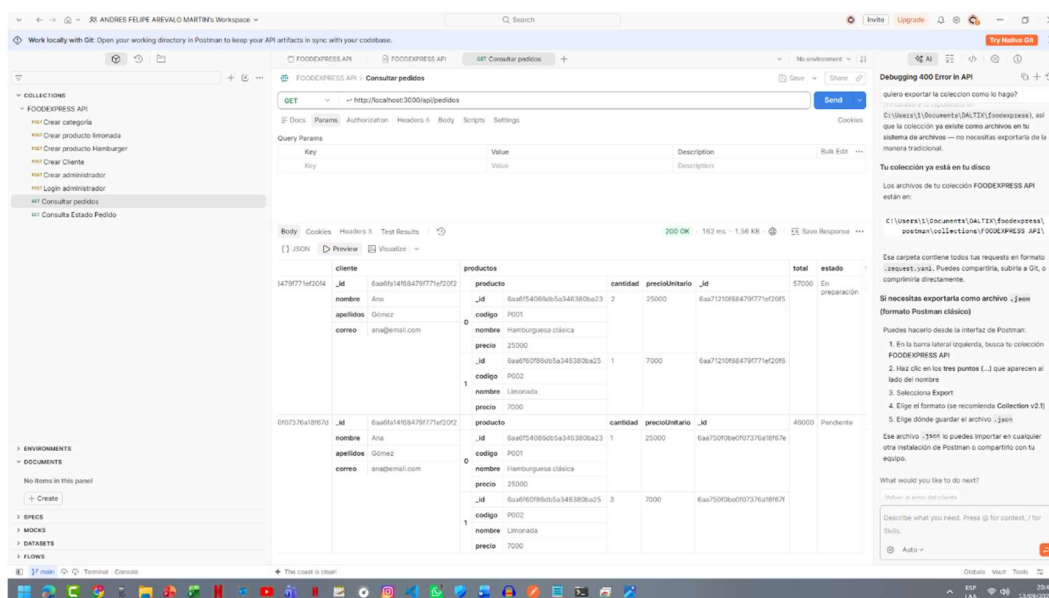


Figura 5. Consulta de pedidos registrados mediante la API REST en Postman.

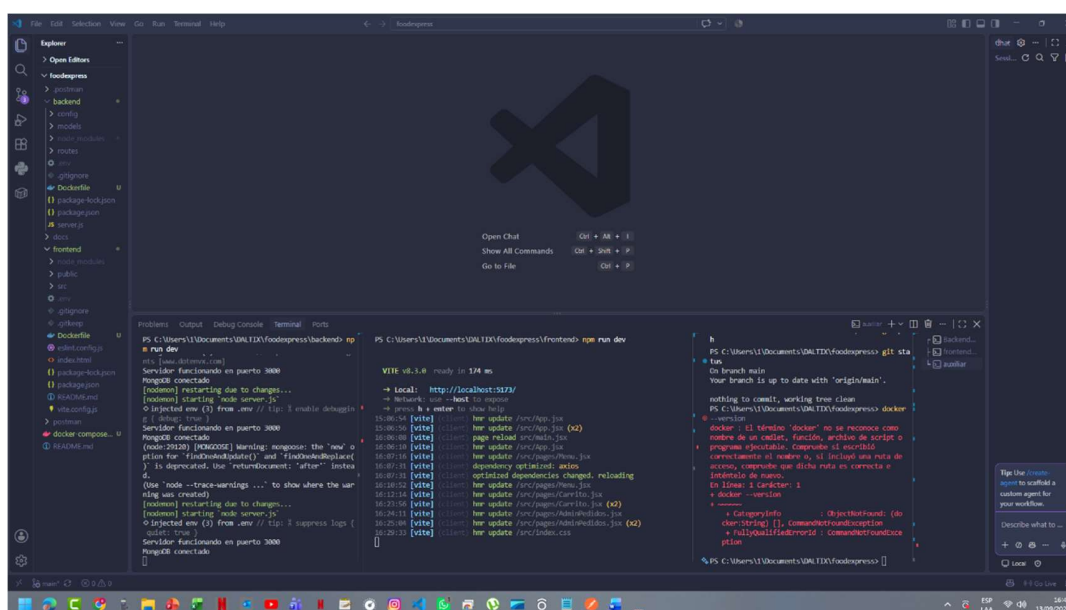


Figura 6. Evidencia del intento de ejecución de Docker en el entorno de desarrollo

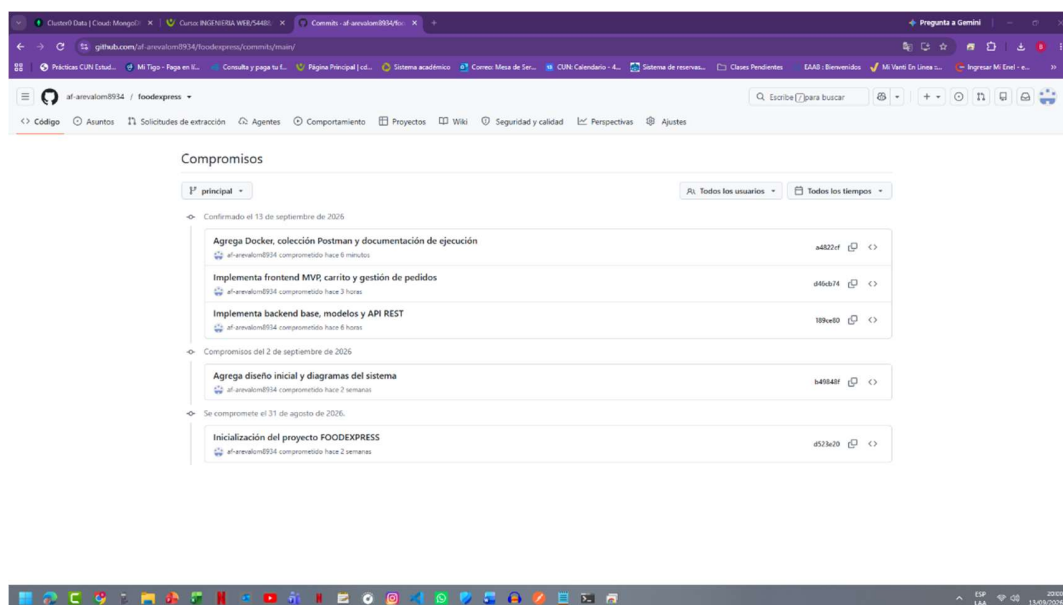


Figura 7. Historial de commits del proyecto FOODEXPRESS en GitHub.

## **Conclusiones**

El desarrollo de FOODEXPRESS permitió implementar y validar un flujo web completo que integra frontend, API REST y persistencia NoSQL. Los productos almacenados en MongoDB son consultados por el backend y visualizados desde React; posteriormente pueden agregarse al carrito y convertirse en un pedido persistente. El módulo administrativo permite consultar estos pedidos y modificar su estado. El proyecto evidencia además el uso de Git/GitHub, Postman y una estructura preparada para contenerización. Como trabajo futuro se plantea ampliar las interfaces administrativas, fortalecer la autenticación y validaciones y completar la ejecución de la infraestructura Docker.